

# nanochat: state of the repo, October 2026

2026-10-07

## Contents

- [Revalidation \(2026-10-07\)](#)
- [Executive summary](#)
- [Part 1: artifacts, cost tiers, reproductions](#)
- [nanochat: artifacts, cost tiers, reproduction gotchas \(as of 2026-10-07\)](#)
  - [TL;DR: the blunt answer](#)
  - [A. Checkpoints and artifacts](#)
  - [B. Cost and time tiers](#)
  - [C. Reproductions and gotchas on rented 8xH100](#)
- [Part 2: changes since launch, fit for LUT substitution](#)
- [nanochat: what changed since launch, and how well it fits a LUT-for-FFN swap](#)
  - [1. What changed](#)
  - [2. Fit for LUT substitution \(current tree\)](#)
- [Part 3: Single-GPU d24 feasibility \(added 2026-10-07\)](#)
  - [3.1 Single-GPU support](#)
  - [3.2 Gradient accumulation and numeric comparability \(the critical question\)](#)
- [819's single-5090 run shows 5,568 steps at 256 accumulation steps, i.e. db5 2.](#)
  - [3.3 Memory on one 80 GB H100](#)
  - [3.4 Wall-clock, cost, and interruption risk](#)
  - [3.5 Evaluation on one GPU](#)
  - [3.6 Upstream reports of 1-GPU \(or few-GPU\) runs](#)
  - [3.7 Which GEMMs run in fp8, and in what precision state is kept \(pending item, now confirmed from code\)](#)
  - [3.8 Corrections made elsewhere in this report by this section](#)
  - [3.9 Confirmed d24 architecture and parameter budget \(read from code, 2026-10-07\)](#)

**As of 2026-10-07.** Upstream HEAD `92d63d4` (2026-07-03). Our local checkout `~/projects/nanochat` is `da32e1d`, from the same day and 9 commits behind.

Sources: the GitHub repo, the commits API, HuggingFace model and dataset pages, and repo discussions. Every claim below has an inline link.

Everything about the current code was checked against the GitHub API, the compare diff and the local tree.

**Revalidated 2026-10-07** with working web access (17 approvals granted, 0 timed out; primary sources fetched directly). The section below lists what was confirmed, what was corrected, and what is still unverified; the body text has been updated where it was wrong.

## Revalidation (2026-10-07)

### Confirmed against primary sources

- Upstream HEAD is still `92d63d4` ("clean up fragile code", 2026-07-03 22:54 UTC); no newer commits ([commits API](#)).
- README: "\$48 (~2 hours of 8XH100 GPU node)", "~\$15" on spot, record 1.65 h, GPT-2 CORE target 0.256525, "GPT-2 capability happens to be approximately depth 26", Lambda recommended; **no \$300 or \$1000 tiers** ([README](#)).

- Leaderboard (in the README; there is no LEADERBOARD.md): #1 d24 3.04 h CORE 0.2585 (Jan 29) → #2 d26+fp8 2.91 h → #3 2.76 h → #4 ClimbMix 2.02 h CORE 0.2571 (**Mar 4 2026**, 324e69c) → #5 1.80 h → #6 **1.65 h, val\_bpb 0.71800, CORE 0.2626** (Mar 14 2026, a825e63).
- runs/speedrun.sh trains **d24** with --depth=24 --target-param-data-ratio=8 --fp8, downloads 170 shards, and is commented "approximately 1.5 hours to complete" ([speedrun.sh](#)).
- Launch speedrun ([discussion #1](#), 2025-10-13): d20, 560,988,160 params, 11,219,763,200 tokens, **3 h 51 min, \$92.40**, CORE 0.2219; report card as in §B.2. Karpathy did not release the d20 checkpoint (community uploads exist, e.g. nanochat-students/nanochat-d20).
- d32 ([discussion #8](#)): 1,879,048,192 params, 37.58 B tokens, ~31.3 h, ~\$800, CORE 0.3168 and the chat scores as in §B; weights at karpathy/nanochat-d32 (SFT chat model, MIT).
- d34 ([model card](#)): base pretrained, 2,217,082,880 params, 88,683,315,200 tokens, CORE 0.3382, ~100 h, ~\$2,500, MIT.
- ChrisMcCormick/decoderstack-d24 ([card](#)): CORE 0.2517, "verified on an H100 (2026-08-01)" with branch varlen-gsm8k at d9dffe9, torch 2.9.1; needs its own 32k tokenizer.

## Corrections

- **The "equally old commit" quote is not Karpathy's.** In [discussion #481](#) it is a comment by co-maintainer @svlandeg (2026-04-20): the d34 "is 5 months old so you'll have to use an equally old commit from this repo to be able to run it". #481 itself is the "\$73 / 3.04 h GPT-2" announcement. Fixed below.
- **The "\$300 / d26 / ~12 h" tier could not be found** in the launch walkthrough (it only says d26 reaches "GPT-2 capability model of CORE about 0.25", no price) nor in the README. It is likely a mis-recollection; removed as a tier. Press coverage of the launch quotes a "~\$1,000 over ~42 hours" tier ([example](#)); the actual d32 run took ~31.3 h / ~\$800.
- **"CORE 0.2626 (5-run mean)":** the README does not say how the record's CORE was averaged; the "5-run mean" is not confirmed and is now marked as such.
- **New community checkpoints found** (none changes the answer): Marcolini/nanochat-d24-base-champion (2026-05-29) and Marcolini/nanochat-d24-base-r24-s615173 (2026-09-08) are d24-depth 910M models from an RTX-4070 fork (Bl4ckd09/nanochat-on-rtx4070), trained on FineWeb-EDU, weights only (no tokenizer), CORE 0.149-0.151 ([card](#)). Not loadable into current master either. The blunt answer stands: **no public base checkpoint loads into current nanochat.**

## Still unverified (not found or not worth an approval prompt)

- Community reproduction reports (Reddit, Hacker News, X): searches returned only press articles, no troubleshooting threads. NCCL/driver/egress gotchas remain from the repo's own code and docs only.
- CORE eval-bundle size, live issue/PR/star counts, the exact contents of 92d63d4, disk budget figures.

- The launch-time code state in §1.3 (reconstructed from the commit trail, not re-read at the launch commit).
- How the record's CORE is averaged (see above).

## Executive summary

**Can we skip pretraining? No.** No public base checkpoint loads into current nanochat.

- Karpathy's two official checkpoints predate the current architecture and tokenizer data. Current code loads with `strict=True`, so both fail:
  - [nanochat-d32](#), Oct 2025, chat (SFT), 7.25 GB, MIT;
  - [nanochat-d34](#), Nov 2025, base, 8.58 GB, MIT, CORE 0.3382.
  - Co-maintainer @svlandeg says d34 needs "an equally old commit" ([discussion #481](#); corrected: this is not Karpathy's quote).
- The \$100 d20 model was never released officially.
- The nearest usable artifact is the community [ChrisMcCormick/decoderstack-d24](#) (CORE 0.2517, 4.2 GB, MIT). It's verified only on a side-branch commit (`d9dffe9`, `varlen-gsm8k`) with its own tokenizer; loading on master is untested.

### Current tiers:

- **Speedrun:** now **d24, about 1.65 h of pretraining** (about 1.5–2.5 h end to end, about \$48 on 8xH100 at Lambda prices), CORE 0.2626 against the GPT-2 bar of 0.2565. The README no longer lists \$300 or \$1000 tiers.
- **Older data points:**
  - d32, Oct 2025: about 31 h, about \$800, CORE 0.3168;
  - d34 base, Nov 2025: about 100 h, about \$2,500, CORE 0.3382.

### Biggest changes since launch:

1. **Dataset:** FineWeb-EDU → NVIDIA ClimbMix-400B (Mar 4). `val_bpb` is not comparable across this point.
2. **Midtraining deleted** (Jan 31). The pipeline is now base → SFT → optional RL.
3. **Architecture:** FA3 with a fallback to standard PyTorch attention; "SSSL" sliding-window attention; value embeddings; per-layer residual scalars; and smear/backout from automated tuning. The MLP stays ReLU<sup>2</sup>, after SwiGLU and MoE were rejected.
4. **Optimizer:** a single MuonAdamW (Polar Express, NorMuon-style, cautious weight decay, MuonEq/Muon+). Gradient clipping removed. Batch size and weight decay scale automatically with parameter count.
5. **Training precision:** FP8 training. Autocast replaced by explicit dtypes. torch pinned to 2.9.1+cu128. HF dependencies dropped and the report card / web UI deleted (July).

**Activity:** no commits since 2026-07-03 (about 3 months). Karpathy still authors the substantive commits, with @svlandeg co-maintaining. It reads as a feature-frozen reference rather than an abandoned one.

**LUT swap:** `nanochat/gpt.py` `class MLP` and `class Block` (hard-coded `MLP(config)`). The main pitfalls:

- `GPT.setup_optimizer` sends every block parameter to Muon, which would silently orthogonalise a 3-D table;
- parameter-count-driven scaling sets tokens, batch size, learning rate and weight decay;
- the meta-device `init_weights()`;
- whole-model `torch.compile`;

- `strict=True` loading;
- FP8 applies only to `nn.Linear`;
- FA3 is fetched over the network.

**Recommendation:** run the **current speedrun once (d24, about 1.5-2.5 h, about \$50)** at HEAD `92d63d4` on 8xH100. That gives a dense baseline matching current code, tokenizer and data. Then fork for the LUT arm. Larger tiers (d32/d34, \$800-\$2,500) aren't worth it before the LUT arm works at d24 or smaller. Pinning a Nov-2025 commit to use the d34 base checkpoint is a fallback, but it gives up nine months of recipe changes.

---

## Part 1: artifacts, cost tiers, reproductions

---

### nanochat: artifacts, cost tiers, reproduction gotchas (as of 2026-10-07)

---

Scope: [github.com/karpathy/nanochat](https://github.com/karpathy/nanochat) and the public artifacts around it, for a project that uses nanochat as a baseline and swaps in lookup-table components.

**How this was researched.** Web pages were read with WebFetch and WebSearch. Code was checked against the local clone `~/projects/nanochat` (shallow, HEAD `da32e1d`, 2026-07-03), which is close to upstream master. Upstream master's newest commit is `92d63d4`, also 2026-07-03, and it is 9 commits past the local HEAD ([commits](#)).

**Coverage.** Section C is built mainly from the repo's own code and docs: searches for community reproduction reports (Reddit, Hacker News, X) returned only press coverage, no troubleshooting threads.

---

### TL;DR: the blunt answer

**Can we skip pretraining with a public base checkpoint that loads into current nanochat master? NO.** No such checkpoint is known to exist at any depth.

- **Official checkpoints are pre-2026 architecture.** Karpathy released two:
  - [d32](#): an SFT model, Oct 2025.
  - [d34](#): a base model, Nov 2025.
  - Both pre-date value embeddings, the `ve_gate` modules, smear, backout and vocab padding.
- **Current code cannot load them.** It loads with `load_state_dict(strict=True)` and only back-fills `resid_lambdas`, `x0_lambdas` and `window_pattern` (local `nanochat/checkpoint_manager.py`).
- **The maintainers say so** (corrected: the comment is by co-maintainer @svlandeg, 2026-04-20, not Karpathy). The d34 "is 5 months old so you'll have to use an equally old commit from this repo to be able to run it" ([discussion #481](#)).
- **Their tokenizer is also wrong for current code.** It was trained on FineWeb-EDU, but current master trains its tokenizer on ClimbMix.
- **Community checkpoints don't fill the gap:**
  - [Sofie/nanochat\\_d14](#) is near-master but small and old (see A.2).
  - [ChrisMcCormick/decoderstack-d24](#) is GPT-2-grade and verified to load into nanochat at commit `d9dffe9` (2026-08-01), but on the `varlen-gsm8k` branch, with its

own tokenizer. This is the only plausible near-current d24 base, and it still needs a hands-on load test.

### Practical options:

1. Pin an old commit (Nov 2025) and use the d34 base: 2.2B params, CORE 0.3382, ~\$2,500 of compute already spent.
2. Take the DecoderStack d24 and test whether it loads into master.
3. Do the run. The current speedrun is short and cheap: about 1.65 h of pretraining, roughly \$48 for the full pipeline at ~\$24/h (README).

**Recommendation for a lookup-table research baseline:** run the current speedrun once (d24, ClimbMix). That gives an exactly-current, self-consistent baseline: same tokenizer, same code, same val shard.

---

## A. Checkpoints and artifacts

### A.1 Directory layout that current code expects

Everything lives under `$NANOCHAT_BASE_DIR`, which defaults to `~/.cache/nanochat` (`nanochat/common.py:get_base_dir`).

```
~/.cache/nanochat/
tokenizer/          tokenizer.pkl, token_bytes.pt
base_data_climbmix/ shard_00000.parquet ... ; LAST shard on disk is used as val
(convention: shard_06542.parquet)
base_data/          legacy FineWeb-EDU dir (fallback only, prints "DATASET UPGRADE
REQUIRED")
eval_bundle/        CORE eval data (unzipped from S3)
identity_conversations.jsonl (SFT identity data)
base_checkpoints/<model_tag>/ model_XXXXXX.pt, meta_XXXXXX.json, optim_XXXXXX_rankN.pt
chatsft_checkpoints/<model_tag>/
chatrl_checkpoints/<model_tag>/
```

### Loading rules in the code:

- `load_model(source)` maps `base`, `sft` and `rl` to the three checkpoint directories above. There is no `mid` source any more: `midtraining` was removed as a separate stage, and `scripts/` has no `mid_train.py` ([checkpoint\\_manager.py](#)).
- `model_tag` defaults to the largest `d<N>` directory, and `step` defaults to the last step found.
- `build_model` does the following, in order:
  1. Strips the `_orig_mod.` prefix from keys.
  2. Patches missing config `window_pattern` to `"L"`.
  3. Patches missing params `resid_lambdas` to 1 and `x0_lambdas` to 0.
  4. Calls `load_state_dict(strict=True)`.
  5. Asserts that the tokenizer vocab size equals `meta["model_config"]["vocab_size"]`.
- **Current architecture** (local `nanochat/gpt.py`):
  - Embeddings: `wte` and `lm_head` padded to a multiple of 64; `value_embeds` (an `nn.ModuleDict`) on alternating layers, with a `ve_gate` per attention layer that has a value embedding.
  - Mixing parameters: `smear_gate`, `smear_lambda`, `backout_lambda`, `resid_lambdas`, `x0_lambdas`.
  - Attention: sliding-window pattern, default `"SSSL"`.

- None of the extra modules is back-filled. So any checkpoint from before about March 14, 2026, when smear and backout came in via [LEADERBOARD Run 6](#), fails strict load on current master unless you write a patch.
- **Checkpoint storage.** Weights are stored in fp32 (README, "Precision / dtype"). So a d24 at 1.38B params has a model file of about 5.5 GB, plus optimizer shards.

## A.2 Per-artifact table

| Artifact                                               | Status                                                                                                   | URL                                                                                                                                                                    | Size                                                                                                                                                   | License   | Loads into current master?                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>d20 speedrun base/mid/SFT/RL (launch, Oct 2025)</b> | <b>Official: does not exist.</b> Karpathy never uploaded the \$100 d20.                                  | —                                                                                                                                                                      | —                                                                                                                                                      | —         | —                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| d20, community (launch-era)                            | Unofficial                                                                                               | <a href="#">nanochat-students/nanochat-d20</a> (0.6B, updated Oct 27 2025); <a href="#">HarleyCooper/nanochat561</a> (561M, Oct 2025; <a href="#">discussion #37</a> ) | UNCERTAIN                                                                                                                                              | UNCERTAIN | <b>No.</b> Oct-2025 architecture and FineWeb tokenizer.                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>d32 (\$1000 tier)</b>                               | <b>Official: SFT only.</b> Base, mid and RL weights were not released ( <a href="#">discussion #8</a> ). | <a href="#">karpathy/nanochat-d32</a> (Oct 16 2025)                                                                                                                    | <code>model_000650.pt</code><br>7.25 GB + <code>meta_000650.json</code> + <code>tokenizer.pkl</code><br>846 kB + <code>token_bytes.pt</code><br>264 kB | MIT       | <b>No.</b> Old architecture (missing <code>value_embeds</code> , <code>ve_gate</code> , smear, backout; unpadded vocab). Needs an Oct-2025 commit. Files go in <code>tokenizer/</code> and <code>chatsft_checkpoints/d32/</code> per its README.                                                                                                                                                                                         |
| <b>d34 base</b>                                        | <b>Official base (pretrained only).</b>                                                                  | <a href="#">karpathy/nanochat-d34</a> (Nov 20 2025)                                                                                                                    | <code>model_169150.pt</code><br>8.58 GB + meta + tokenizer                                                                                             | MIT       | <b>No on master.</b> @svlandeg (#481): "5 months old so you'll have to use an equally old commit" (#481). Its README says to put it under <code>chatsft_checkpoints/d34/</code> even though it is a base model. That is UNCERTAIN; most likely just for <code>chat_cli</code> . For training, put it in <code>base_checkpoints/d34/</code> . Specs: 2.2B params, 88.7B tokens (ratio 40), CORE 0.3382, val bpb 0.7045, ~100 h, ~\$2,500. |
| d34 SFT (community, HF-transformers port)              | Unofficial                                                                                               | <a href="#">pankajmathur/nanochat-d34-sft-hf</a> (Dec 9 2025)                                                                                                          | ~2B                                                                                                                                                    | UNCERTAIN | No (HF format).                                                                                                                                                                                                                                                                                                                                                                                                                          |

| Artifact                                    | Status                                              | URL                                                                   | Size                                                                                                | License | Loads into current master?                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HF transformers NanoChat class              | Official in transformers, by Ben Burtenshaw         | <a href="#">docs, burtenshaw/nanochat</a>                             | —                                                                                                   | —       | Converts the old-architecture d32 into HF format. Not usable with nanochat code.                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>d14 base + SFT</b>                       | Unofficial, by repo co-maintainer Sofie (@svlandeg) | <a href="#">Sofie/nanochat_d14</a> (about Jan 2026)                   | 2.25 GB total: base/, sft/, tokenizer (tokenizer.pkl 412 kB)                                        | MIT     | <b>Probably not on current master. UNCERTAIN, needs a test.</b> Built at commit 5019acc and stated "runs on current master" as of Jan 31 2026 (#481). It predates the March smear/backout change, which is not back-filled, and the ClimbMix tokenizer switch. Trained on FineWeb-EDU with --window-pattern=L, 399M params, 1.15B tokens, CORE 0.1590, ChatCORE 0.2404.                                                                                                             |
| <b>d24 base, GPT-2 grade (DecoderStack)</b> | Unofficial                                          | <a href="#">ChrisMcCormick/decoderstack-d24</a>                       | nanochat-format model_005568.pt 4.2 GB + 8 optim shards of ~1.0 GB; native capture 2.8 GB + 11.1 GB | MIT     | <b>Closest candidate.</b> Ready-made base_checkpoints/d24_decoders tack/ loads into nanochat's varlen-gsm8k branch, verified at commit d9dffe9 (2026-08-01). Master compatibility is not stated, so UNCERTAIN. Trained with a different (non-nanochat) training stack on ClimbMix: 5.84B tokens, ratio 8, CORE 0.2517, val bpb 0.737 (best-fit). Its 32k tokenizer is <b>different</b> from other d24 releases ("31,474 of 32,759 merge ids differ"), so it needs its own base dir. |
| d24 base/chat on a single consumer GPU      | Unofficial                                          | <a href="#">Marcolini/nanochat-d24-base-champion</a> (+ chat variant) | 2.66 GB                                                                                             | MIT     | Needs that fork's GPT class. 910M params, 9 heads (non-standard width), FineWeb-EDU, CORE 0.1514. Not a useful baseline.                                                                                                                                                                                                                                                                                                                                                            |
| d24 base/SFT/RL (RTX 5090 port)             | Unofficial                                          | <a href="#">Nekochu/nanochat-d24</a>                                  | UNCERTAIN                                                                                           | MIT     | Port: safetensors, "~2,000 lines" rewrite. 1.38B params; GSM8K 16.5% after RL. Not upstream format.                                                                                                                                                                                                                                                                                                                                                                                 |

| Artifact                                                 | Status                                                                                                      | URL                                                                                                                                                         | Size                                                                                                                                                                    | License                                            | Loads into current master?                                                                                                                                 |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| d26 base/SFT/RL ("pretraining priors" control arm)       | Unofficial                                                                                                  | <a href="#">jkminder/pretraining-priors-d26-base (+ -sft, -rl)</a>                                                                                          | 973M params, bf16                                                                                                                                                       | CC BY-NC 4.0 weights                               | No. HF <code>trust_remote_code</code> format; speedrun mechanisms ablated (only logit softcap kept). ClimbMix 7.35B tokens, CORE 0.2489, 8xH200, Aug 2026. |
| JAX port checkpoints                                     | Unofficial                                                                                                  | <a href="#">tucan9389/nanochat-jax</a>                                                                                                                      | —                                                                                                                                                                       | —                                                  | No.                                                                                                                                                        |
| <b>Tokenizer (current, ClimbMix-trained)</b>             | <b>No official upload.</b> It is trained by <code>scripts.tok_train</code> in the speedrun (a few minutes). | —                                                                                                                                                           | ~540 kB on disk (local <code>~/.cache/nanochat/tokenizer</code> )                                                                                                       | —                                                  | Must match the checkpoint. The d32 and d34 tokenizers are FineWeb-trained.                                                                                 |
| <b>Pretraining data, current: ClimbMix-400B shuffled</b> | <b>Official</b>                                                                                             | <a href="#">karpathy/climbmix-400b-shuffle</a> (from <a href="#">nvidia/Nemotron-ClimbMix</a> ); downloaded by <code>python -m nanochat.dataset -n N</code> | 6,543 parquet shards (shard_00000 to shard_06542). Measured locally: 5 shards = 439 MB, so <b>~88 MB/shard</b> . Speedrun needs ~150, downloads 170, so <b>~15 GB</b> . | UNCERTAIN (NVIDIA's ClimbMix license; not checked) | Raw text parquet, tokenized on the fly by the dataloader. <b>Not pretokenized.</b>                                                                         |
| Pretraining data, legacy: FineWeb-EDU-100B shuffled      | Official (legacy)                                                                                           | <a href="#">karpathy/fineweb-edu-100b-shuffle</a>                                                                                                           | 1,823 shards                                                                                                                                                            | ODC-By (FineWeb-EDU); UNCERTAIN for the repack     | Fallback only ( <code>base_data/</code> ). Used until Mar 4 2026.                                                                                          |
| Pretokenized GPT-2-token FineWeb-EDU shards              | Official, but for llm.c, not nanochat                                                                       | <a href="#">karpathy/fineweb-edu-100B-gpt2-token-shards</a>                                                                                                 | —                                                                                                                                                                       | —                                                  | No. GPT-2 tokenizer; nanochat does not consume it.                                                                                                         |
| <b>Midtraining mixture</b>                               | <b>Stage removed.</b> It was folded into SFT (see the SFT row).                                             | —                                                                                                                                                           | —                                                                                                                                                                       | —                                                  | —                                                                                                                                                          |

| Artifact                | Status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | URL                                                                                                                                                                                                                                       | Size                                               | License                 | Loads into current master?                                                                                                              |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>SFT mixture</b>      | Assembled at runtime from HF datasets, not one bundled file. Mixture (local <code>scripts/chat_sft.py</code> ): <a href="#">HuggingFaceTB/smol-smoltalk</a> train (460K) + identity conversations ×2 epochs (1K rows, from <a href="https://karpathy-public.s3.us-west-2.amazonaws.com/identity_conversations.jsonl">https://karpathy-public.s3.us-west-2.amazonaws.com/identity_conversations.jsonl</a> ) + <a href="#">cais/mmlu</a> auxiliary_train ×3 epochs (100K each) + <a href="#">openai/gsm8k</a> main train ×4 (8K each) + SimpleSpelling/SpellingBee (80K synthetic; word list from GitHub dwyl/english-words). | —                                                                                                                                                                                                                                         | small                                              | per source              | n/a                                                                                                                                     |
| RL data                 | GSM8K train, GRPO-style ( <code>scripts/chat_rl.py</code> , loads <code>sft</code> , writes <code>chatrl_checkpoints</code> )                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | —                                                                                                                                                                                                                                         | —                                                  | —                       | n/a                                                                                                                                     |
| <b>CORE eval bundle</b> | <b>Official</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <a href="https://karpathy-public.s3.us-west-2.amazonaws.com/eval_bundle.zip">https://karpathy-public.s3.us-west-2.amazonaws.com/eval_bundle.zip</a> , auto-downloaded by <code>scripts/base_eval.py</code> into <code>eval_bundle/</code> | UNCERTAIN (size not verified). 22 DCLM CORE tasks. | DCLM/component licenses | Yes, current.                                                                                                                           |
| ChatCORE tasks          | From HF at eval time: ARC ( <a href="#">allenai/ai2_arc</a> ), MMLU, GSM8K, HumanEval ( <a href="#">openai/openai_humaneval</a> ), SpellingBee                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | —                                                                                                                                                                                                                                         | —                                                  | —                       | Yes. Note: newer upstream commit <a href="#">a5a3a33</a> "delete datasets dependency" changes how these are fetched; details UNCERTAIN. |

### A.3 Runtime network dependencies that are not data shards

These matter on a locked-down box or inside the `sbox` cage.

- **FA3 kernel.** It is fetched from the HF Kernels Hub at import time:  
`kernels.get_kernel("varunneal/flash-attention-3")` on Hopper, or `kernels-community/flash-attn3` (local `nanochat/flash_attention.py`). Without it, training falls back to SDPA, which is much slower and has no fast sliding window. So cache it in `~/.cache/huggingface` beforehand.
  - **Other downloads during the run:** the eval bundle (S3), identity conversations (S3), the SpellingBee word list (GitHub), and the HF datasets above.
-

## B. Cost and time tiers

### B.1 What the repo states now (master, Oct 2026)

| Tier                                           | Depth                                                                                                                | Params                                                         | Train tokens                                                                                                                                                      | FLOPs                                         | Pretrain wall clock                                                     | Full pipeline                                                                                                                    | \$                                                            | CORE                                                                                                                                  | Chat metrics                                                                                                             |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Speedrun, current</b><br>(runs/speedrun.sh) | <b>d24</b> , fp8, ClimbMix, --target-param-data-ratio=8                                                              | ~1.38B total ( <a href="#">#481</a> ; 1,384,124,976 at Jan 29) | ratio 8 × scaling params (729,810,624) → <b>5,838,471,168 tokens</b> = 5,568 steps × 1,048,576 (computed from HEAD code, Part 3; corrected 2026-10-07 from "~7B") | ~4e19 ("4e19 FLOPs capability model", README) | <b>1.65 h</b> (99 min; <a href="#">LEADERBOARD Run 6</a> , Mar 14 2026) | "~1.5 hours" (speedrun.sh header; README). Tokenizer + pretrain + base_eval + SFT + chat_eval. RL is <b>not</b> in the speedrun. | "\$48 (~2 hours of 8XH100)" at ~\$24/h; spot "~\$15" (README) | <b>0.2626</b> (averaging not stated in the README); GPT-2 bar 0.256525. val bpb 0.71800 (ClimbMix, not comparable to FineWeb numbers) | <b>Not published for the current speedrun.</b> UNCERTAIN; no current report card was found in the README or LEADERBOARD. |
| Leaderboard history                            | d24 (Jan 29) → d26 fp8 (Feb 2) → d26 1M batch (Feb 5) → d24 ClimbMix (Mar 4) → autoresearch r1 (Mar 9) → r2 (Mar 14) | —                                                              | —                                                                                                                                                                 | 4.33e19 (Run 1)                               | 3.04 → 2.91 → 2.76 → 2.02 → 1.80 → 1.65 h                               | —                                                                                                                                | Run 1 ~\$73                                                   | 0.2585, 0.2578, 0.2602, 0.2571, 0.2690, 0.2626                                                                                        | —                                                                                                                        |
| Miniseries v1 (Jan 7 2026, FineWeb-EDU era)    | d10-d20, compute-optimal (ratio ~8 at the time)                                                                      | d10 91M, d12 135M, d15 229M, d20 477M                          | 0.73 / 1.08 / 1.83 / 3.82 B                                                                                                                                       | —                                             | 5 / 7.8 / 17.4 / 59.5 min                                               | whole sweep ~4 h                                                                                                                 | ~\$100 for the whole sweep                                    | 0.071 / 0.1059 / 0.1158 / 0.1708                                                                                                      | — ( <a href="#">#420</a> )                                                                                               |
| d12 quick-iteration scale                      | d12                                                                                                                  | ~135M                                                          | —                                                                                                                                                                 | —                                             | "~5 min pretraining runs" (README "Research")                           | —                                                                                                                                | —                                                             | —                                                                                                                                     | —                                                                                                                        |

| Tier                                           | Depth | Params        | Train tokens                                              | FLOPs | Pretrain wall clock                                                                                                    | Full pipeline | \$       | CORE                                      | Chat metrics                                                                                                                                                                                       |
|------------------------------------------------|-------|---------------|-----------------------------------------------------------|-------|------------------------------------------------------------------------------------------------------------------------|---------------|----------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>\$1000 tier (d32), launch-era, official</b> | d32   | 1,879,048,192 | 37,580,963,840                                            | —     | ~31 h                                                                                                                  | —             | ~\$800   | base CORE 0.3168                          | Mid → SFT → RL: ARC-E 0.6233 → 0.6797; ARC-C 0.4787 → 0.4991; MMLU 0.3896 → 0.4049; GSM8K 0.1099 → 0.1274 → <b>0.1994 (RL)</b> ; HumanEval 0.1098 → 0.1280; ChatCORE 0.2417 → 0.2734 ( <b>#8</b> ) |
| <b>~\$2,500 tier (d34 base), official</b>      | d34   | 2.2B          | 88.7B (ratio 40, "2X longtrained compared to Chinchilla") | —     | ~100 h                                                                                                                 | —             | ~\$2,500 | <b>0.3382</b> , val bpb 0.7045, MFU 47.4% | — ( <b>HF card</b> )                                                                                                                                                                               |
| ~~\$300 tier (d26), launch-era~~               | d26   | —             | —                                                         | —     | NOT FOUND (corrected 2026-10-07): the launch post states no price for d26, only "GPT-2 capability ... CORE about 0.25" | —             | —        | —                                         | —                                                                                                                                                                                                  |

The README no longer lists \$300 or \$1000 tiers. The only scripted tier is the speedrun; the old `run1000.sh` no longer exists in `runs/`.

## B.2 Launch numbers (Oct 13 2025), for comparison. Re-fetched and confirmed 2026-10-07

Source: [discussion #1](#), which the README itself now calls "deprecated information... a lot older (with worse results)".

**Run setup:** d20, 560,988,160 params, 11,219,763,200 tokens, FineWeb-EDU, 3 h 51 min on 8xH100, \$92.40. Pipeline: base → mid → SFT → (optional) RL.

### Report card:

| Metric    | Base   | Mid    | SFT    | RL     |
|-----------|--------|--------|--------|--------|
| CORE      | 0.2219 | —      | —      | —      |
| ARC-E     | —      | 0.3561 | 0.3876 | —      |
| ARC-C     | —      | 0.2875 | 0.2807 | —      |
| MMLU      | —      | 0.3111 | 0.3151 | —      |
| GSM8K     | —      | 0.0250 | 0.0455 | 0.0758 |
| HumanEval | —      | 0.0671 | 0.0854 | —      |
| ChatCORE  | —      | 0.0730 | 0.0884 | —      |

## B.3 Changes since launch (verified from repo docs)

- **Speedrun target and model changed.**
  - Target: the old "~\$100 / 4 h d20 ChatGPT" is now "time to GPT-2 CORE (0.256525)".
  - Model: a d24 at about 1.4B params. Compared with the old d20, that is about 2.5× the params, CORE 0.26 vs 0.22, and under half the cost.
- **Dataset:** FineWeb-EDU-100B was replaced by ClimbMix-400B on Mar 4 2026. That gave -27% time, and **val\_bpb is not comparable across the switch** ([LOG](#)).
- **Pipeline:**
  - Midtraining was removed as a separate stage; SFT absorbs MMLU and GSM8K.
  - Since Feb 16 2026, SFT runs whole-epoch with optimizer warm-start and ChatCORE every 200 steps.
- **Architecture:** sliding-window attention, value embeddings, per-layer residual and x0 scalars, smear and backout (from autoresearch), padded vocab, fp8 ( --fp8; first via torchao, since 2026-02-10 nanochat's own `nanochat/fp8.py`, see §1.3), and auto batch-size scaling (d26 uses a 1M-token batch).
- **Precision:** autocast was removed. An explicit `COMPUTE_DTYPE` was added, with fp32 master weights.

---

## C. Reproductions and gotchas on rented 8xH100

Limited: community-report searches (Reddit, HN, issues) were cut short by the tool gating. Items below come from repo code and docs unless linked.

| Topic                         | Finding                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Software stack                | Pinned torch==2.9.1 from the <b>cu128</b> index (local pyproject.toml). No torchao: fp8 is nanochat's own nanochat/fp8.py (torchao was dropped on 2026-02-10, \$1.3; it is not in pyproject.toml at da32e1d or HEAD; corrected 2026-10-07). kernels>=0.11.7. Python ≥3.10. Set up with uv sync --extra gpu (speedrun installs uv itself). The driver must support CUDA 12.8, i.e. R570 or later (UNCERTAIN: general CUDA requirement, not stated in the repo). |
| Flash attention               | FA3 comes from the HF Kernels Hub, needs sm90 for the best kernel, and needs network at import. Without it, training silently falls back to SDPA, which is much slower and gives no sliding-window speedup; the docs recommend --window-pattern L in that case. FA3 supports bf16/fp8 only.                                                                                                                                                                    |
| fp8                           | Needs H100-class hardware and torch.compile; fp8 without compile is ~4× slower. The capability-matched gain is only ~5%. On GPUs without fp8, drop --fp8 (bf16 gives a slightly stronger model).                                                                                                                                                                                                                                                               |
| Data download                 | nanochat.dataset -n 8 first (tokenizer training), then -n 170 in the background during tokenizer training. That is ~170 × 88 MB ≈ <b>15 GB</b> from HF (measured shard size locally). HF egress is free to you, but throughput varies by provider.                                                                                                                                                                                                             |
| Disk                          | Rough budget, partly UNCERTAIN: ~15 GB data, plus a ~5.5 GB fp32 d24 model, plus optimizer shards (DecoderStack's d24: 8 × ~1.0 GB), plus SFT checkpoints, plus a uv venv of several GB. <b>≥100 GB free is comfortable</b> . Checkpoints at launch-era scale: d32 7.25 GB, d34 8.58 GB (model only).                                                                                                                                                          |
| Memory                        | Default --device-batch-size is 32. The speedrun uses 16 for d24 (gradient accumulation compensates). On GPUs with less than 80 GB, reduce it.                                                                                                                                                                                                                                                                                                                  |
| Nondeterminism / eval noise   | Training is "mildly non-deterministic". 7 identical d24 ClimbMix runs gave CORE 0.2512–0.2677 (spread 0.0165, mean 0.2571). Data shard order also matters, and the default order is an "unlucky" one ( <b>LEADERBOARD Run 4</b> ). <b>Use val_bpb (much less noisy) for comparisons within one dataset. If you use CORE, use multi-seed averages.</b>                                                                                                          |
| Wall clock                    | Pretraining 1.65 h (iterations only, excluding eval and logging). Script total is quoted as ~1.5 h; plan 2–2.5 h including uv sync, data download, CORE eval and SFT. UNCERTAIN: no community end-to-end timing was collected.                                                                                                                                                                                                                                 |
| Providers                     | README: "I use and like Lambda". 8xH100 at "~\$3/GPU/hr ≈ \$24/hr", spot "~\$15" total. 8xA100 "will run just fine... a bit slower".                                                                                                                                                                                                                                                                                                                           |
| Single GPU                    | Works without torchrun (automatic gradient accumulation) and takes ~8× longer. Community runs exist: d24 on an RTX 5090 in ~3.8 days for pretraining ( <b>Nekochu</b> ), and <b>discussion #819</b> "GPT-2 training on RTX 5090" (Aug 2026, not read).                                                                                                                                                                                                         |
| Cage / offline note (gpustar) | The run needs network for: HF shards, the HF Kernels Hub FA3 kernel, HF task datasets for SFT and eval, S3 eval_bundle and identity jsonl, and GitHub for the word list. Pre-download into ~/.cache (nanochat base dir and HF cache) before a no-network run. This box already has 5 ClimbMix shards plus a tokenizer in ~/.cache/nanochat.                                                                                                                    |

**Not covered (web gated mid-task):** NCCL issues, provider-specific failures, egress costs, HN, Reddit, X threads, the issue tracker.

## Part 2: changes since launch, fit for LUT substitution

### nanochat: what changed since launch, and how well it fits a LUT-for-FFN swap

Research date: 2026-10-07. Upstream: <https://github.com/karpathy/nanochat> (branch master). Local checkout: a local clone @ da32e1d (2026-07-03, shallow clone, clean working tree).

**How this was gathered.** Commit history came from the GitHub REST API (/commits?per\_page=100&page=1..5). The local→HEAD delta came from the compare API: <https://api.github.com/repos/karpathy/nanochat/compare/da32e1d...92d63d4>. Code facts come from reading the local da32e1d tree plus that compare diff. Dated rationale comes from dev/LOG.md and dev/LEADERBOARD.md in the checkout.

**Limits.** Two things were not re-checked against primary sources:

- the code as it stood at launch (§1.3 reconstructs it from the commit trail);
- live issue and PR counts.

Those items are marked UNCERTAIN. Line links point at `da32e1d`, which differs from HEAD only by the 9 commits listed in §1.2.

## 1. What changed

### 1.1 Where our checkout sits

- **Launch.** The oldest commits on API page 5 are dated 2025-10-14, e.g. `67aaca9` (<https://github.com/karpathy/nanochat/commit/67aaca9>). The public launch was about 2025-10-13. The initial commit was not retrieved (UNCERTAIN on the exact sha).
- **Local `da32e1d`.** Dated 2026-07-03, message "remove a ton of UI bloat ... nukes ~1000 LOC" (<https://github.com/karpathy/nanochat/commit/da32e1d>). It deleted `chat_web.py`, the web UI and the report machinery.
- **Upstream HEAD.** `92d63d4`, also 2026-07-03, "clean up fragile code" (<https://github.com/karpathy/nanochat/commit/92d63d4>).
- **Gap.** Our checkout is only **9 commits behind HEAD**, all from the same day. Our checkout has roughly 9 months of changes since launch; the user's mental model (launch-era, Oct 2025) does not. The real knowledge gap is launch → July 2026, not local → HEAD.
- **Size.** About 430–460 commits in total: 4 full pages of 100 plus about 30 on page 5. The GitHub page fetch reported 441, but that page looked stale (UNCERTAIN).

### 1.2 The 9 commits between local `da32e1d` and HEAD `92d63d4`

All 9 are dated 2026-07-03 and authored by karpathy. Source:

<https://api.github.com/repos/karpathy/nanochat/compare/da32e1d...92d63d4>

| Commit                                                                                                                                            | What it did                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Relevance to us                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>950a1dc</code><br>( <a href="https://github.com/karpathy/nanochat/commit/950a1dc">https://github.com/karpathy/nanochat/commit/950a1dc</a> ) | Deletes "cute" bloat: the <code>dev/gen_synthetic_data.py</code> identity-data generator, the <code>customjson.py</code> task, and <code>curl identity_conversations.jsonl</code> in <code>runs/*.sh</code> .                                                                                                                                                                                                                                                                                                                | Low                                                                                                   |
| <code>4186540</code><br>( <a href="https://github.com/karpathy/nanochat/commit/4186540">https://github.com/karpathy/nanochat/commit/4186540</a> ) | Removes HuggingFace. <code>HuggingFaceTokenizer</code> is deleted from <code>nanochat/tokenizer.py</code> ; only <code>RustBPETokenizer</code> ( <code>rustbpe train + tiktoken inference</code> ) remains. <code>base_eval.py</code> loses <code>--hf-path</code> (no more evaluating HF models such as GPT-2). The <code>tokenizers</code> and <code>transformers</code> deps are dropped.                                                                                                                                 | Medium: we can no longer score an HF GPT-2 checkpoint with <code>base_eval</code>                     |
| <code>a5a3a33</code><br>( <a href="https://github.com/karpathy/nanochat/commit/a5a3a33">https://github.com/karpathy/nanochat/commit/a5a3a33</a> ) | Removes the <code>datasets</code> dependency. New <code>tasks/common.py::HubDataset / load_hub_dataset()</code> downloads parquet via the HF Hub HTTP API with <code>pyarrow</code> and <code>filelock</code> , cached at <code>{base_dir}/task_data/...</code> ARC, GSM8K, MMLU, SmolTalk and HumanEval are ported to it. <code>tasks/spellingbee.py</code> is deleted, so <code>SpellingBee</code> leaves SFT and <code>chat_eval</code> . Deps added: <code>numpy</code> , <code>pyarrow</code> , <code>filelock</code> . | <b>Medium: ChatCORE drops from 6 to 5 tasks, so its numbers are not comparable across this commit</b> |
| <code>f527f76</code><br>( <a href="https://github.com/karpathy/nanochat/commit/f527f76">https://github.com/karpathy/nanochat/commit/f527f76</a> ) | Drops the <code>torchao</code> mention from the <code>--fp8</code> help text.                                                                                                                                                                                                                                                                                                                                                                                                                                                | –                                                                                                     |

| Commit                                                                                                                               | What it did                                                                                                                                                                                                                                                                                                             | Relevance to us                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| f4f69f9<br>( <a href="https://github.com/karpathy/nanochat/commit/f4f69f9">https://github.com/karpathy/nanochat/commit/f4f69f9</a> ) | Rewrites the <code>execution.py</code> sandbox (HumanEval) from multiprocessing to a subprocess with a GUARD prelude (−289 / +74 lines).                                                                                                                                                                                | Low                                                                                                                         |
| a9d0a86<br>( <a href="https://github.com/karpathy/nanochat/commit/a9d0a86">https://github.com/karpathy/nanochat/commit/a9d0a86</a> ) | <b>Unifies the optimizer.</b> <code>DistMuonAdamW</code> is deleted and the single <code>MuonAdamW</code> skips collectives when <code>world_size==1</code> . <code>gpt.py</code> now always does <code>optimizer = MuonAdamW(param_groups)</code> .                                                                    | <b>High: there is now only one optimizer class to patch for LUT params</b>                                                  |
| f8a85a5<br>( <a href="https://github.com/karpathy/nanochat/commit/f8a85a5">https://github.com/karpathy/nanochat/commit/f8a85a5</a> ) | Adds pytest suites <code>tests/test_optim.py</code> , <code>test_tasks.py</code> , <code>test_tokenizer.py</code> , <code>test_execution.py</code> .                                                                                                                                                                    | Useful: <code>test_optim</code> is a template for testing a LUT parameter group                                             |
| eb16d01<br>( <a href="https://github.com/karpathy/nanochat/commit/eb16d01">https://github.com/karpathy/nanochat/commit/eb16d01</a> ) | Adds <code>scripts/infer_bench.py</code> (prefill/decode, MFU and MBU). Adds <code>GPT.num_matmul_params()</code> , <code>estimate_decode_flops()</code> , <code>estimate_prefill_flops()</code> , <code>kv_bytes_per_token()</code> and <code>kv_read_bytes()</code> . Adds <code>common.get_peak_bandwidth()</code> . | <b>High: a ready-made harness for the efficiency argument (decode bandwidth and FLOPs), relevant to our efficiency case</b> |
| 92d63d4<br>( <a href="https://github.com/karpathy/nanochat/commit/92d63d4">https://github.com/karpathy/nanochat/commit/92d63d4</a> ) | "clean up fragile code". Details not read; it may touch the FLOP and param counting via <code>num_matmul_params</code> (UNCERTAIN).                                                                                                                                                                                     | Check before patching                                                                                                       |

Dependencies at HEAD are `kernels`, `psutil`, `rustbpe`, `tiktoken`, `torch==2.9.1`, `wandb`, `filelock`, `numpy`, `pyarrow`. The gpu extra uses the **cu128** wheel index, and the dev group no longer includes `transformers`. Sources: [pyproject @ da32e1d](#) and the compare diff above.

### 1.3 Dated list of the important changes since launch

Each entry: date, short sha (commit link), then what changed.

#### Oct 2025 (launch month).

The launch-time code state below is reconstructed from the commit trail, not re-read at the launch commit (UNCERTAIN in detail); the recipe numbers are confirmed against [discussion #1](#):

- **Recipe:** d20 (560,988,160 params) via `speedrun.sh`, 3 h 51 min on 8×H100 for \$92.40.
- **Data:** FineWeb-EDU-100B shards (`karpathy/fineweb-edu-100b-shuffle`).
- **Tokenizer:** `rustbpe`, 65,536 vocab.
- **Architecture:** RoPE, QK-norm, untied embeddings, ReLU<sup>2</sup> MLP, parameter-free RMSNorm, no biases, MQA/GQA option, logit softcap 15, PyTorch SDPA attention.
- **Optimizer:** Muon (matrices) plus a separate DistAdamW (embeddings and `lm_head`).
- **Pipeline:** base → **midtraining** → SFT → optional RL on GSM8K. `report.md` "report card". Web UI. Calculator tool use in the Engine.

Changes during the month:

- 2025-10-16 306bc38 (<https://github.com/karpathy/nanochat/commit/306bc38>) and 786119d (<https://github.com/karpathy/nanochat/commit/786119d>): CPU and MPS support with device autodetect.
- 2025-10-21 a088b7a (<https://github.com/karpathy/nanochat/commit/a088b7a>): uses SDPA `enable_gqa`.

- 2025-10-21 fe5aed9 (<https://github.com/karpathy/nanochat/commit/fe5aed9>): "personality" / identity data for SFT.
- 2025-10-24 8892470 (<https://github.com/karpathy/nanochat/commit/8892470>): SpellingBee task. Deleted again on 2026-07-03.

## Nov 2025.

- 2025-11-13 c6abcdf (<https://github.com/karpathy/nanochat/commit/c6abcdf>): **pretraining resumption** (optimizer and dataloader state in checkpoints).
- 2025-11-13 7b7fd0f (<https://github.com/karpathy/nanochat/commit/7b7fd0f>): thanks svlandeg, who became the co-maintainer.
- 2025-11-15 bc1fca3 (<https://github.com/karpathy/nanochat/commit/bc1fca3>): MQA renamed to GQA.
- Late Nov: assorted fixes to the KV cache, find\_last\_step, and an fp32 cast before the softcap (cbf30c8, <https://github.com/karpathy/nanochat/commit/cbf30c8>, merged 2025-12-08).

## Dec 2025.

- 2025-12-23 bc51da8 (<https://github.com/karpathy/nanochat/commit/bc51da8>): **pad vocab to a multiple of 64**. lm\_head and wte now have the padded shape, and logits are cropped.
- 2025-12-27/28: tf32 API change and a model\_tag cleanup.

## Jan 2026: the big rewrite month.

Infrastructure and dependencies:

- 2026-01-01 48abd7d (<https://github.com/karpathy/nanochat/commit/48abd7d>): init simplified (uniform init with zero-init c\_proj).
- 2026-01-03 aa42f40 (<https://github.com/karpathy/nanochat/commit/aa42f40>): the inline Rust BPE project is removed. rustbpe is now a pip dependency.
- 2026-01-04 9c60dfb (<https://github.com/karpathy/nanochat/commit/9c60dfb>): **torch pinned to 2.9.1**.
- 2026-01-04 eb7bbc1 (<https://github.com/karpathy/nanochat/commit/eb7bbc1>): **configurator replaced by argparse**. CLI flags changed (breaking for old command lines).

Recipe and scaling:

- 2026-01-05 ae0bf52 (<https://github.com/karpathy/nanochat/commit/ae0bf52>): hyperparameters tuned from sweeps; "warmdown\_ratio is the biggest free win".
- 2026-01-07 cc4b7f (<https://github.com/karpathy/nanochat/commit/cc4b7f>): miniseries and scaling laws (runs/scaling\_laws.sh, runs/miniseries.sh).
- 2026-01-08 061f83c (<https://github.com/karpathy/nanochat/commit/061f83c>): **grad clip deleted**.

Optimizer:

- 2026-01-11 2c4473d (<https://github.com/karpathy/nanochat/commit/2c4473d>): **Muon upgrades** following modded-nanogpt: Polar Express, factored variance reduction (NorMuon-style), cautious weight decay.

Architecture:

- 2026-01-11 aa530cd (<https://github.com/karpathy/nanochat/commit/aa530cd>): **per-layer resid\_lambdas / x0\_lambdas**, with x0 blended back into the residual at every layer.
- 2026-01-11 201d705 (<https://github.com/karpathy/nanochat/commit/201d705>): **checkpoint patching** for the new lambdas.
- 2026-01-11 2ff7d51 (<https://github.com/karpathy/nanochat/commit/2ff7d51>): **Flash Attention 3** via the HF `keras` package (+9% tok/s at d12).
- 2026-01-11 fbc1484 (<https://github.com/karpathy/nanochat/commit/fbc1484>) and b33e394 (<https://github.com/karpathy/nanochat/commit/b33e394>): **sliding-window pattern "SSSL"**. Short window =  $\frac{1}{4}$  context; the last layer is always full.

Data and tokenizer:

- 2026-01-13 43c29dd (<https://github.com/karpathy/nanochat/commit/43c29dd>): **big DataLoader refactor**. BOS-aligned "best-fit" packing with epoch and resume state.
- 2026-01-13 64b48d0 (<https://github.com/karpathy/nanochat/commit/64b48d0>): tokenizer regex `\p{N}{1,2}` validated.
- 2026-01-15 6bb9240 (<https://github.com/karpathy/nanochat/commit/6bb9240>) and 22a71aa (<https://github.com/karpathy/nanochat/commit/22a71aa>): Muon and AdamW each become a single `torch.compile` fused step.

More architecture:

- 2026-01-16 8203efa (<https://github.com/karpathy/nanochat/commit/8203efa>): **FA3 with SDPA fallback** (`nanochat/flash_attention.py`).
- 2026-01-16 9a88194 (<https://github.com/karpathy/nanochat/commit/9a88194>) and e85db6b (<https://github.com/karpathy/nanochat/commit/e85db6b>): **Value embeddings (ResFormer)**: one VE per layer, then alternating layers. Merged as branch `ve` on 2026-01-18 (63bb583, <https://github.com/karpathy/nanochat/commit/63bb583>).
- 2026-01-17 413e91a (<https://github.com/karpathy/nanochat/commit/413e91a>): optimal tokens:params ratio found to be about 4 at that time. It was re-derived later; see Mar 24.
- 2026-01-18 a91743c (<https://github.com/karpathy/nanochat/commit/a91743c>): `.sh` scripts moved into `runs/`.
- 2026-01-27/28 c8d93be (<https://github.com/karpathy/nanochat/commit/c8d93be>) then 74554be (<https://github.com/karpathy/nanochat/commit/74554be>): **Engram-lite bigram hash-embedding tables added, then reverted**. They helped at d12 but the gain vanished at d25 on wall-clock, and VRAM bloated. This is the closest prior art to a LUT layer in this repo; see `dev/LOG.md` "2026-01-27".
- 2026-01-29 41bb2ea (<https://github.com/karpathy/nanochat/commit/41bb2ea>): **AdamW and Muon combined into a single MuonAdamW** driven by `kind='adamw'|'muon'` param groups (plus `DistMuonAdamW`).
- 2026-01-29: **leaderboard Run 1**. d24 reaches GPT-2 CORE in 3.04 h (**LEADERBOARD**; commit 348fbb3, <https://github.com/karpathy/nanochat/commit/348fbb3>).
- 2026-01-31 1ddaad1 (<https://github.com/karpathy/nanochat/commit/1ddaad1>): **midtraining deleted** ("nuke midtraining from orbit"). The pipeline is now `base` → `SFT` (→ optional `RL`).
- 2026-01-31 3c3a3d7 (<https://github.com/karpathy/nanochat/commit/3c3a3d7>): `warmdown 0.5`.

**Feb 2026.**

- 2026-02-01 0307997 (<https://github.com/karpathy/nanochat/commit/0307997>): `base_loss` and `base_eval` merged into a single `scripts/base_eval.py`.

- 2026-02-01 dc291c6 (<https://github.com/karpathy/nanochat/commit/dc291c6>): **Blackwell sm100 via the SDPA fallback.**
- 2026-02-02 433aacf (<https://github.com/karpathy/nanochat/commit/433aacf>): improved FA3 kernel loading.
- 2026-02-03 6079f78 (<https://github.com/karpathy/nanochat/commit/6079f78>) and a67eba3 (<https://github.com/karpathy/nanochat/commit/a67eba3>): **FP8 training** (torchao, tensorwise). Leaderboard Run 2: d26, 2.91 h.
- 2026-02-05 f41dd3c (<https://github.com/karpathy/nanochat/commit/f41dd3c>) and 2c062aa (<https://github.com/karpathy/nanochat/commit/2c062aa>): **auto-computed optimal batch size** (Power Lines,  $B \propto D^{0.383}$ ) and weight decay scaling. Run 3: 2.76 h.
- 2026-02-05 d63b7ab (<https://github.com/karpathy/nanochat/commit/d63b7ab>): SwiGLU tried and failed. ReLU<sup>2</sup> stays.
- 2026-02-10 e569b59 (<https://github.com/karpathy/nanochat/commit/e569b59>): **torchao removed; own nanochat/fp8.py Float8Linear.**
- 2026-02-16 788dade (<https://github.com/karpathy/nanochat/commit/788dade>) and 8180e1d (<https://github.com/karpathy/nanochat/commit/8180e1d>): SFT upgrades. Warmup/warmdown schedule, **optimizer warm-start from base (default on)**, ChatCORE eval during SFT, and MMLU×3 / GSM8K×4 epochs.
- 2026-02-19 2dffdc8 (<https://github.com/karpathy/nanochat/commit/2dffdc8>): **MoE (DeepSeekV3-style) documented as a negative result.** Per-step it was better; on wall-clock it was worse.

## Mar 2026.

- 2026-03-02: logit softcap sweep, 20 best per LOG. Note that the code at da32e1d has `softcap = 15` ([gpt.py L470](#)); it was presumably re-tuned later (UNCERTAIN).
- 2026-03-04 1076f97 (<https://github.com/karpathy/nanochat/commit/1076f97>): **autocast deleted.** A global `COMPUTE_DTYPE` plus a custom `Linear` that casts fp32 master weights to the activation dtype; fp16 + GradScaler path.
- 2026-03-04 324e69c (<https://github.com/karpathy/nanochat/commit/324e69c>): **BREAKING: dataset FineWeb-EDU-100B → NVIDIA ClimbMix-400B** (karpathy/climbmix-400b-shuffle, 6543 parquet shards, dir `base_data_climbmix/`). Leaderboard Run 4: d24, 2.02 h. val\_bpb is not comparable to earlier runs.
- 2026-03-09 6ed7d1d (<https://github.com/karpathy/nanochat/commit/6ed7d1d>): **"autoresearch round 1"**, changes found by Claude running autonomously at d12. Run 5: 1.80 h.
- 2026-03-14 a825e63 (<https://github.com/karpathy/nanochat/commit/a825e63>): **autoresearch round 2.** `smear` (previous-token embedding mix), `backout` (subtract the mid-layer residual) and hyperparameter tuning. **Run 6 = current record: 1.65 h, val\_bpb 0.71800, CORE 0.2626.**
- 2026-03-24 1cd94d7 (<https://github.com/karpathy/nanochat/commit/1cd94d7>): default `--target-param-data-ratio` set to 12.
- 2026-03-24 4e1694c (<https://github.com/karpathy/nanochat/commit/4e1694c>): parameter-golf ideas tried, all negative (LeakyReLU<sup>2</sup>, partial RoPE, ...).
- 2026-03-25 c0dbf1f (<https://github.com/karpathy/nanochat/commit/c0dbf1f>): `COMPUTE_DTYPE`-aware Muon cast.
- 2026-03-26 03be953 (<https://github.com/karpathy/nanochat/commit/03be953>) and a445144 (<https://github.com/karpathy/nanochat/commit/a445144>): dependencies trimmed and a dev dependency group added.

## Apr-May 2026: quiet.

- 2026-04-13 9822cc7 (<https://github.com/karpathy/nanochat/commit/9822cc7>) and 94b73ad (<https://github.com/karpathy/nanochat/commit/94b73ad>): smear and backout lambdas are now actually initialised in `init_weights`. This was a meta-device bug.
- 2026-05-05 dc54a1a (<https://github.com/karpathy/nanochat/commit/dc54a1a>): DyT tried and failed.

## Jul 2026: a "minimal and forkable" cleanup burst, then silence.

- 2026-07-02 f10bd75 (<https://github.com/karpathy/nanochat/commit/f10bd75>): **report card deleted**.
- 2026-07-02 fbb50ad (<https://github.com/karpathy/nanochat/commit/fbb50ad>): rename `fwe` → `climbmix`.
- 2026-07-03 4e014d3 (<https://github.com/karpathy/nanochat/commit/4e014d3>): **MuonEq row equilibration + Muon+ Frobenius renorm** added to Muon.
- 2026-07-03 ca366eb (<https://github.com/karpathy/nanochat/commit/ca366eb>): KV cache allocated in `COMPUTE_DTYPE`.
- 2026-07-03 da32e1d (<https://github.com/karpathy/nanochat/commit/da32e1d>): web UI deleted (**our local checkout**).
- 2026-07-03: the 9 commits in §1.2, ending at HEAD 92d63d4.

## 1.4 Current state by area, at HEAD / da32e1d

**Recipe.** One dial, `--depth. model_dim = depth × aspect_ratio(64)`, rounded up to a multiple of `head_dim=128`, and `n_head = model_dim/128` ([base\\_train.py build\\_model\\_meta L129-143](#)).

- The training horizon is `target_param_data_ratio × scaling_params`, with a default of 12. Here `scaling_params` = transformer block matrices + `lm_head`; wte and the value embeddings are excluded (§3.9). Speedrun uses `--depth=24 --target-param-data-ratio=8 -fp8` ([runs/speedrun.sh](#)).
- Batch size is auto-computed from the d12 reference ( $B_{REF} = 2^{19}$ ) via  $D^{0.383}$ . LR scales with  $\sqrt{(B/B_{ref})}$ , and weight decay with  $\sqrt{(B/B_{ref}) \cdot (D_{ref}/D)}$ .
- The LR schedule is linear warmup (40 steps), constant, then linear warmdown over 65% of the run to a final  $0.05\times$ . Muon momentum is also scheduled, as is weight decay ([base\\_train.py L359-395](#)).
- Seq len 2048. Vocab 32768 ( $2^{15}$ ) ([tok\\_train.py](#)), padded to a multiple of 64.

## Architecture ([nanochat/gpt.py](#)):

- **Attention:** RoPE with base 100000, and QK-norm scaled  $\times 1.2$ . GQA is supported, but nanochat sets `n_kv_head = n_head`. FA3 is used via `keruels` with an SDPA fallback. Sliding-window pattern "SSSL" (**short window 512** for  $T=2048$ , last layer full; corrected 2026-10-07: the code computes `-(-2048 // 4 // 128) * 128 = 512`, and the inline comment "2048 -> 768" is stale, see §3.9). Value embeddings with a sigmoid gate on alternating layers (ResFormer).
- **MLP and norms:** ReLU<sup>2</sup> MLP with a  $4\times$  expansion: two matrices, no gate, no bias ( $8d^2$  params per block; §3.9). Parameter-free RMSNorm (`F.rms_norm`), applied pre-attn, pre-MLP, after the embedding and before `lm_head`.
- **Embeddings and head:** untied wte and `lm_head`, no biases, logit softcap 15 computed in fp32.
- **Extra scalars:** per-layer `resid_lambdas` and `x0_lambdas`, smear gate, backout lambda.
- **Precision:** fp32 master weights; bf16 compute via `COMPUTE_DTYPE`; embeddings stored in bf16. Optional FP8 via [nanochat/fp8.py](#) on all `nn.Linear` layers with `dims % 16 == 0` and `min dim ≥ 128` (so the gates are skipped).

**Optimizer.** A single `MuonAdamW` ([nanochat/optim.py](#)).

- Muon: Nesterov momentum → MuonEq → Polar Express (5 iterations) → Muon+ renorm → NorMuon-style factored variance reduction → cautious weight decay. It runs on all `transformer.h` params, stacked by shape.
- AdamW: `lm_head`, `wte`, value embeddings and scalars, each with its own LR, betas and weight decay.
- Since `a9d0a86`, single-GPU and DDP share one class.

**Tokenizer.** `rustbpe` (training) + `tiktoken` (inference) only. The HF tokenizer path was deleted in `4186540`.

## Data.

- Pretraining uses `ClimbMix-400B` parquet shards downloaded from <https://huggingface.co/datasets/karpathy/climbmix-400b-shuffle> ([nanochat/dataset.py L23](#)). About 170 shards for speedrun.
- The dataloader is `tokenizing_distributed_data_loader_with_state_bos_bestfit` (BOS-aligned best-fit packing, resumable).
- SFT data: `SmolTalk` + `MMLU` (`auxiliary_train`) ×3 + `GSM8K` ×4. Identity JSON and `SpellingBee` were removed in `950a1dc` / `a5a3a33`.

## Eval.

- Base: `val_bpb` (vocab-invariant bits per byte, [nanochat/loss\\_eval.py](#)), plus the DCLM **CORE** score (22 ICL tasks, centered accuracy, eval bundle downloaded lazily) in [scripts/base\\_eval.py](#) `evaluate_core`.
- Chat: [scripts/chat\\_eval.py](#). **ChatCORE** is the mean centered accuracy over ARC-Easy, ARC-Challenge, MMLU, GSM8K and HumanEval. `SpellingBee` was dropped at HEAD.

## Post-training.

- Midtraining was **removed** (2026-01-31).
- [scripts/chat\\_sft.py](#) warm-starts the base optimizer state by default.
- [scripts/chat\\_rl.py](#) is simplified "GRPO" on GSM8K: on-policy, no KL, no PPO clip, `advantage = r - mean`. It is no longer in `speedrun.sh`.

**Inference.** [nanochat/engine.py](#) has a `KVCache` that also carries `prev_embedding` for smear. It uses `FA3 flash_attn_with_kvcache` or the SDPA path, and the calculator tool (`<|python_start|> ... use_calculator`). The CLI is `scripts/chat_cli.py`; the web UI is gone. `scripts/infer_bench.py` was added at HEAD.

**Dependencies.** `torch==2.9.1` from the `cu128` index; Python ≥ 3.10; `kernels` for FA3. Our `gpustar` stack is `torch cu130 / CUDA 13.3`, so the pinned `torch==2.9.1+cu128` will conflict with our spiky environment. Use a separate venv or relax the pin.

## Breaking changes to note:

- `configurator` → `argparse` (Jan 4).
- Checkpoints gained `resid/x0 lambdas` (patched on load), `VE`, `smear` and `backout`. Old checkpoints are **not** loadable for smear/backout; there is no patch for those. UNCERTAIN beyond what `_patch_missing_keys` shows: [checkpoint\\_manager.py L22-39](#) only patches `lambdas` and `window_pattern`.
- Vocab padding (Dec 23).

- `DistAdamW` → `MuonAdamW` → `DistMuonAdamW` removed, which changes the optimizer state dict format.
- Dataset dir `base_data/` → `base_data_climbmix/`.
- `midtrain` removed; `base_loss` merged into `base_eval`; `HuggingFaceTokenizer` removed; `chat_web` removed.

## 1.5 Activity and maintenance

### Commit cadence:

- Very high in Oct 2025 – Mar 2026, often many commits a day.
- Low in Apr – May 2026: PR merges on Apr 13-14, one DyT log on May 5.
- A burst of about 25 commits on 2026-07-02/03.
- **Then nothing: zero commits from 2026-07-04 to the research date (2026-10-07), about 3 months.** Source: API page 1 tops out at 92d63d4 (2026-07-03), [https://api.github.com/repos/karpathy/nanochat/commits?per\\_page=100](https://api.github.com/repos/karpathy/nanochat/commits?per_page=100).

### Maintainers.

- Karpathy authored every substantive commit through HEAD.
- Sofie Van Landeghem (@svlandeg) acts as co-maintainer, merging PRs and fixing docs (thanked in 7b7fd0f).
- There is no evidence of a hand-off. The repo was not archived as of the GitHub page fetch (UNCERTAIN: that page looked cached at Mar 2026).

**Issues, PRs and size.** The page reported 28 open issues, 95 open PRs, about 58.5k stars and 8.2k forks. All UNCERTAIN / stale; re-check live.

**Reading.** The July 2026 commits explicitly reposition nanochat as "small and sexy and minimal and forkable ... code is ~free now" (da32e1d). It looks like a feature-frozen reference harness, which is good for a fork baseline. Karpathy's research energy moved to `karpathy/autoresearch`, referenced in [LEADERBOARD Run 5](#).

## 2. Fit for LUT substitution (current tree)

### 2.1 Where things live

**FFN.** `class MLP` in `nanochat/gpt.py` (L131-141 @ da32e1d).

- `c_fc = Linear(n_embd, 4n)` and `c_proj = Linear(4n, n_embd)`.
- `forward = c_proj(relu(c_fc(x)).square())`.
- `Linear` is nanochat's own subclass that casts fp32 weights to the activation dtype (L45-50).

**Block.** `class Block` (L144-153):

- `self.mlp = MLP(config)`, hard-coded with no factory or config switch;
- `x = x + attn(norm(x), ve, cos_sin, window_size, kv_cache)`;
- `x = x + mlp(norm(x))`.

The input to the MLP is a parameter-free RMSNorm of the residual stream.

**Residual plumbing around blocks.** `GPT.forward` (L453-466):

- `x = resid_λ[i]·x + x0_λ[i]·x0` before each block;

- the backout subtraction at layer  $n/2$ ;
- smear before the trunk.

**Config.** `@dataclass GPTConfig` (L28-39) has `sequence_len`, `vocab_size`, `n_layer`, `n_head`, `n_kv_head`, `n_embd`, `window_pattern`.

- It is built in `scripts/base_train.py::build_model_meta(depth)` (L129-143).
- It is saved into checkpoint meta as `asdict(config)` and rebuilt with `GPTConfig(**meta["model_config"])` in `checkpoint_manager.build_model` (L76-115).
- **New LUT fields must have defaults**, and any new CLI flag must be threaded into `build_model_meta`.

**Init.** `GPT.__init__` runs **under** `torch.device("meta")`; the docstring calls this "a major footgun" (L157-162). Then `model.to_empty(device)` and `model.init_weights()` (base\_train L146-151).

- `init_weights` initializes every tensor explicitly **by attribute name** (`block.mlp.c_fc.weight`, `block.mlp.c_proj.weight`, L226-232).
- A LUT module will hit an `AttributeError` there. Worse, any tensor it does not explicitly init stays **uninitialised garbage** from `to_empty`. The smear/backout bug fixed in 94b73ad was exactly this.
- Buffers computed in `__init__` (random hyperplanes, index maps, bit-weight vectors) are meta tensors with no data and must be recomputed in `init_weights`. Rotary embeddings do the same (L257-260).

**Optimizer grouping.** `GPT.setup_optimizer` (L376-416). At HEAD it is the same except `MuonAdamW(param_groups)` unconditionally (a9d0a86).

- `matrix_params = list(self.transformer.h.parameters())`: **every parameter inside the blocks goes to Muon**, grouped by `p.shape`.
- The AdamW groups are `lm_head`, `wte`, `value_embeds`, `resid`, `x0` and `smear`, each with its own `lr`, `betas` and `weight_decay`. The LR is scaled by  $(d/768)^{-0.5}$ .
- An `assert` checks that the param counts across groups add up to `len(self.parameters())`.

## 2.2 Why this is awkward for LUT tables

### 1. Muon would silently orthogonalise LUT tables.

- `muon_step_fused` (optim.py L109-178) treats the last two dims as a matrix. MuonEq row-norm, Polar Express, the Muon+ renorm to  $\sqrt{\min(m,n)}$  and NorMuon variance reduction all act per matrix.
- A 3-D table (`n_tables`, `2^k`, `d_out`) would be stacked to 4-D and orthogonalised per table **without error**, which is exactly the wrong update. The MoE experiment in LOG 2026-02-19 relied on this broadcast behaviour.
- A 1-D parameter (thresholds, biases, temperatures) would crash: `.mT / size(-2)`.
- The Muon LR also includes a `max(1, rows/cols)^0.5` shape multiplier, and cautious weight decay is 0.28 scaled.
- **Fix:** split `transformer.h` params into a LUT group and a matrix group. Put the LUT group in a new `kind='adamw'` group (or a custom kind) with its own LR and weight decay, and update the param-count assert.
- Note that attention in the same block keeps Muon, so a hybrid block needs a filter (e.g. by module type or a `p._is_lut` tag) rather than "all of `transformer.h`".

**2. DDP AdamW sharding constraint.** For AdamW params with  $\geq 1024$  elements, `_reduce_adamw` uses `reduce_scatter` and **asserts** `shape[0] % world_size == 0` (optim.py L401-

417).

- LUT tables need a leading dim divisible by 8 for 8-GPU runs.
- This is irrelevant on single-GPU (5090); after `a9d0a86`, `world_size==1` skips the collectives.

### 3. Param and FLOP accounting drives the training recipe.

- `num_scaling_params()` counts **all transformer.h params** as `transformer_matrices` (L347-374).
- `base_train` uses `transformer_matrices + lm_head` to set:
  - the **number of training tokens** (`target_param_data_ratio × params`);
  - the **auto batch size** ( $D^{0.383}$ , relative to a d12 reference built with the same model class!);
  - the **LR batch scaling**;
  - the **weight-decay scaling** (`base_train` L263-306).
- Large LUT tables would inflate the horizon and batch, so they would train on more tokens than the dense baseline. Small tables would do the reverse.
- `estimate_flops()` counts  $6 × (\text{all params} - \text{embeddings} - \text{scalars})$  as matmul FLOPs (L319-345), so MFU and the `--target-flops` path become wrong.
- **Fix:**
  - For iso-token comparison with the baseline, pass `--num-iterations` and `--total-batch-size` explicitly.
  - Otherwise, exclude LUT params from both functions.
  - HEAD adds `num_matmul_params()` in `eb16d01`, the natural hook to adjust.

### 4. torch.compile.

- `base_train` wraps the whole model: `torch.compile(model, dynamic=False)` (L245-246). So does `chat_sft` (L120).
- The optimizer steps are `@torch.compile(fullgraph=True)`.
- A LUT forward with custom CUDA, Triton, `argsort`, data-dependent shapes or Python-side control flow will graph-break or recompile.
- Options: wrap the kernel as a `torch.library` custom op (or `@torch._dynamo.allow_in_graph` with a custom `autograd.Function`, as `fp8.py` does), or keep the op purely tensorised with gather/index ops.
- CORE eval and sampling run on the **uncompiled** `orig_model`.

### 5. Precision.

- Activations are bf16 (`COMPUTE_DTYPE`) and master weights fp32.
- Only nanochat's `Linear` casts weights at use (L45-50), so a LUT must do its own `.to(x.dtype)` on table reads.
- Embeddings are stored in bf16 (`init_weights` L262-268); decide deliberately whether tables should be fp32 masters.
- Hard thresholds or comparisons in bf16 can flip; consider fp32 for the addressing math.
- `--fp8` converts only `nn.Linear` with `dims % 16 == 0` and  $≥ 128$  (`fp8_module_filter`, `base_train` L175-189). LUT params are skipped, which is fine, but an fp8 baseline vs a bf16 LUT is not apples-to-apples. Compare without `--fp8`.
- `disable_fp8()` finds modules by `'Float8'` in `type(m).__name__`.

### 6. Attention assumptions on our hardware (RTX 5090, sm\_120).

- FA3 is loaded at import via the HF `kernels` package, a **network download** ([flash\\_attention.py L23-48](#)). Hopper uses `varunneal`; others use `kernels-community` if available.
- On `sm_120`, and inside `sbox` (no network), expect the **SDPA fallback**.
- With the default `SSSL` window pattern, SDPA must build an **explicit boolean mask** for the short-window layers ([L99-110](#)). That is a slower, non-flash path.
- For FFN-only research on the 5090, consider `--window-pattern L` for both baseline and LUT, which keeps everything on `is_causal=True` flash SDPA. It does change the baseline, so it is not comparable to the leaderboard.
- `_compute_window_sizes` rounds to FA3 tiles of 128.

## 7. Checkpoint save/load.

- Save is `orig_model.state_dict() + optimizer.state_dict()` (per-rank file) + meta JSON with `model_config` ([base\\_train L478-491](#)).
- Load is `GPT(GPTConfig(**meta))` on meta → `to_empty` → `init_weights()` → `load_state_dict(strict=True, assign=True)` ([checkpoint\\_manager L96-104](#)).
- Consequences:
  - A) LUT checkpoints load fine **in a LUT-aware fork**, provided the config carries the LUT hyperparameters (with defaults) and persistent buffers are in the state dict. Non-persistent buffers must be recomputed in `init_weights()`, which `build_model` calls before loading.
  - B) `strict=True` means a dense checkpoint cannot be loaded into a LUT model (for distillation or initialising a LUT from a trained MLP) without a custom loader, and vice versa.
  - C) `assign=True` replaces tensors with the loaded ones (dtype included).
  - D) **The optimizer state is tied to param-group layout.** `chat_sft` loads the base optimizer state **by default** (`--load-optimizer=1`, LOG 2026-02-16). Resume (`--resume-from-step`) and SFT warm-start break if groups differ between producer and consumer.
  - E) On CPU/MPS, `build_model` casts `bf16` → `fp32`.

## 8. Inference engine.

- `Engine` with `KVCache` is position-wise for the MLP, so a stateless per-token LUT needs no engine changes.
- `kv_cache` is threaded through `Block.forward` only for attention.
- `GPT.generate` (naive) and `Engine` are cross-checked by `tests/test_engine.py`, which is a free correctness test for a LUT swap.

## 9. Hard-coded MLP references elsewhere.

- `init_weights` (by name).
- `fp8_module_filter` (implicitly).
- `estimate_flops` and `num_scaling_params` (implicitly, via all params).
- The docstring table in `init_weights`.
- `grep -n "mlp" nanochat scripts` before patching. `chat_sft.py` / `chat_rl.py` rebuild models only via `checkpoint_manager`, so they inherit the fix.

## 10. Prior art in this repo.

- Engram-lite bigram hash tables (LOG 2026-01-27/28): plain `nn.Embedding` tables, **AdamW with the embedding LR**, zero-init, per-layer  $\lambda$  injection. Helped at d12, gone at d25 on wall-clock; reverted.

- MoE (LOG 2026-02-19): a 3-D expert tensor under Muon; per-step better, wall-clock worse.
- Both show that Karpathy scores by **wall-clock time to CORE**, not per-step loss. For our purpose (energy and deployability), report per-step/per-token val\_bpb **and** the `infer_bench.py` FLOPs and bytes, not just the leaderboard metric.

## 2.3 What the eval harness measures, and baselines

**val\_bpb.** `evaluate_bpb(model, batches, steps, token_bytes)` ([nanochat/loss\\_eval.py](#)) gives bits per byte on the ClimbMix val split.

- It runs every `--eval-every=250` steps over `--eval-tokens=80×524288` ( $\approx 42\text{M}$  tokens).
- It is tokenizer-invariant and is Karpathy's recommended smooth metric.
- `token_bytes` was fixed to use raw token bytes on 2026-07-03 (2ce972a, <https://github.com/karpathy/nanochat/commit/2ce972a>). **Do not compare val\_bpb across that commit, or across the Mar 4 dataset switch.**

**CORE.** DCLM CORE: 22 ICL tasks (ARC, HellaSwag, PIQA, Winograd, LAMBADA, BoolQ, ...), centered accuracy averaged. Via `evaluate_core` in [scripts/base\\_eval.py](#); the GPT-2 1.5B reference is 0.256525. Noisy: about  $\pm 0.008$  run-to-run, with a 7-run spread of 0.0165 ([LEADERBOARD Run 4](#)).

**ChatCORE.** Centered mean over ARC-E, ARC-C, MMLU (random baseline 0.25), GSM8K and HumanEval (baseline 0). At HEAD this is 5 tasks; at `da32e1d` it is 6 with SpellingBee ([scripts/chat\\_eval.py L204-241](#)).

### Baseline numbers (8×H100, ClimbMix):

| Run                                          | Setup             | Time   | val_bpb | CORE                                        |
|----------------------------------------------|-------------------|--------|---------|---------------------------------------------|
| Run 6 (current record), <code>a825e63</code> | d24, ratio 8, fp8 | 1.65 h | 0.71800 | 0.2626 (averaging not stated in the README) |
| Run 5                                        | d24, ratio 8.7    | 1.80 h | 0.71808 | 0.2690                                      |
| Run 4                                        | d24, ratio 9.5    | 2.02 h | 0.71854 | 0.2571 (7-run mean 0.25714)                 |
| Pre-ClimbMix (FineWeb-EDU; not comparable)   | d26               | –      | 0.74645 | 0.2602                                      |

**What is missing.** There are **no published d12/d16 val\_bpb numbers** for the current code. For a 5090-scale study we must produce our own d12 (or d16/d20) dense baseline at the same commit, flags and token budget (UNCERTAIN whether `dev/scaling_analysis.ipynb` lists per-depth numbers; not checked).

## 2.4 Bottom line

The FFN is a 10-line MLP class with one instantiation site (`Block.__init__`), so the swap itself is trivial. The friction is entirely in the surrounding machinery:

1. `setup_optimizer` sends everything in `transformer.h` to Muon;
2. `num_scaling_params` / `estimate_flops` treat all block params as matmul weights, and this drives tokens, batch, LR and weight decay;
3. meta-device init plus name-based `init_weights`;
4. whole-model `torch.compile`;
5. `strict=True` loading and optimizer-state coupling (SFT warm-start);
6. FA3/SDPA-with-mask behaviour on `sm_120` without network;

7. the torch 2.9.1+cu128 pin vs our cu130 stack.

Fork at HEAD 92d63d4 (single optimizer class, `infer_bench`, `num_matmul_params`). Add a `GPTConfig.mlp_type` field (default "relu2") and an `mlp_factory`. Tag LUT params and route them to a dedicated AdamW group. Exclude them from scaling and FLOP counts, or pin `--num-iterations / --total-batch-size`. Run the baseline and LUT arms at the same depth without `--fp8`.

---

## Part 3: Single-GPU d24 feasibility (added 2026-10-07)

---

**Question.** Can the d24 speedrun baseline be reproduced on **one H100 80GB** instead of 8×H100, with a number that is comparable to the 8-GPU run?

**Verdict: yes, with caveats.**

- The code supports one GPU natively.
- The global batch, step count, learning rates and weight decay are derived from parameter count and token budget, **never from GPU count**. So a 1-GPU run follows the same optimisation trajectory, up to data order and floating-point noise.
- Expect ~13-18 h end to end, at about the same GPU-hours (and so about the same dollars) as 8×H100.

**Sources.** Code read at HEAD 92d63d4 (the local clone was fast-forwarded from da32e1d for this section). The d24 numbers were computed by building the model on the meta device from that code. Line links point at 92d63d4. Upstream reports come from the [README](#) and GitHub discussions.

### 3.1 Single-GPU support

- **Entry points accept 1 GPU.**
  - `base_train.py` documents `python -m scripts.base_train` (plain python) next to `torchrun` ([base\\_train.py L1-12](#)).
  - `compute_init` only initialises NCCL when `RANK/LOCAL_RANK/WORLD_SIZE` are set ([common.py L137-209](#)). So both plain python and `torchrun --nproc_per_node=1` work; the latter creates a 1-rank process group.
- `runs/speedrun.sh` **hardcodes** `--nproc_per_node=8` for `base_train`, `base_eval`, `chat_sft` and `chat_eval`. For one GPU, those four lines must be edited to `--nproc_per_node=1` (or plain python `-m ...`).
- **No DDP/FSDP wrapping.** The model is never wrapped. Gradient sync happens inside the optimiser.
- **The optimiser handles one rank natively.** Since a9d0a86 (one of the 9 commits between da32e1d and HEAD) there is a single `MuonAdamW` class: "The same class handles both single GPU and distributed training" ([optim.py L1-6](#)).
  - Multi-rank: ZeRO-2-style sharding. AdamW uses `reduce_scatter/all_gather` for large params; Muon divides the stacked matrices across ranks ([optim.py L203-250](#)).
  - World size 1: every communication op is skipped and the rank owns all params ([optim.py L274-301](#), [optim.py L402-405](#), [optim.py L429-436](#)).
  - The divisibility assert (`shape[0] % world_size`) only applies to multi-rank runs.
- **Muon's maths does not depend on sharding.** Every operation in `muon_step_fused` is per matrix:

- momentum;
  - MuonEq row equilibration;
  - Polar Express;
  - Muon+ renormalisation;
  - factored variance reduction;
  - cautious weight decay. These use norms over dims (-2, -1) or a single reduction dim ([optim.py L112-180](#)). Sharding only decides *which rank* computes which matrix.
- **The repo's own optimiser tests are single-rank.** They contain no multi-rank test ([tests/test\\_optim.py](#)).

## 3.2 Gradient accumulation and numeric comparability (the critical question)

What is derived from GPU count, and what is not.

- **Independent of GPU count:**
  - target tokens = ratio × scaling params ([base\\_train.py L263-269](#));
  - the auto batch size, from tokens via the Power-Lines rule ([base\\_train.py L276-284](#));
  - the LR scale  $\sqrt{(B/B\_ref)}$  ([base\\_train.py L286-294](#));
  - weight decay  $\lambda \cdot \sqrt{(B/B\_ref) \cdot (D\_ref/D)}$  ([base\\_train.py L296-304](#));
  - the iteration count, target\_tokens // total\_batch\_size ([base\\_train.py L348-351](#));
  - the LR, momentum and weight-decay schedules (functions of step only, [base\\_train.py L359-386](#)).
- **Dependent on GPU count:** only `grad_accum_steps = total_batch_size // (device_batch_size × max_seq_len × world_size)` ([base\\_train.py L406-413](#)). This keeps the global batch invariant.

Gradient semantics are the same.

- Each micro-step's loss is divided by `grad_accum_steps` before `.backward()` ([base\\_train.py L510-518](#)).
- Multi-rank runs average across ranks (`ReduceOp.AVG`).
- Either way, the gradient is the mean over the same 1,048,576-token global batch.

**d24 at HEAD, computed from the code** (vocab 32,768, the `tok_train` default):

| Quantity                                       | Value                                                                                                  |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Total params                                   | 1,384,122,122 (wte 50.3M, value embeds 604.0M, lm_head 50.3M, transformer matrices 679.5M, 74 scalars) |
| Scaling params (matrices + lm_head)            | 729,810,624                                                                                            |
| Target tokens (ratio 8)                        | 5,838,484,992                                                                                          |
| Auto total batch                               | 1,048,576 tokens ( $2^{20}$ )                                                                          |
| Iterations                                     | 5,568 → 5,838,471,168 tokens trained                                                                   |
| FLOPs/token<br>( <code>estimate_flops</code> ) | 4.775e9 → 2.79e19 total                                                                                |
| grad_accum, 8 GPUs, dbs 16<br>(speedrun)       | 4                                                                                                      |
| grad_accum, 1 GPU, dbs 16 / 8 / 2              | 32 / 64 / 256                                                                                          |

These match upstream:

- the 5,568 steps match decoderstack-d24's `model_005568.pt`;
- the 1.384B params match #481;

## • 819's single-5090 run shows 5,568 steps at 256 accumulation steps, i.e. dbs 2.

---

**What differs between a 1-GPU and an 8-GPU run.** Nothing in the hyperparameters. What differs is noise of the same kind as seed-to-seed variation:

1. **Data order.** The loader strides parquet row groups by rank: rank  $r$  reads groups  $r, r+W, \dots$  ([dataloader.py L52-68](#)). So the documents in a given step, and the best-fit packing into rows, differ from the 8-GPU run. The token *distribution* and total are the same; the exact batches are not.
2. **Floating-point reduction order** (gradient accumulation vs NCCL AVG), plus non-deterministic kernels. These are not bitwise reproducible even across two 8-GPU runs.
3. **fp8 tensorwise scales** are computed per micro-batch tensor (amax). Keeping `--device-batch-size=16` gives the same micro-batch shape as the speedrun. A smaller dbs changes scale granularity slightly. It is mathematically harmless but not identical.

The README states the same: one GPU "will produce ~identical results (code will automatically switch to gradient accumulation), but you'll have to wait 8 times longer" ([README L82](#)).

**Conclusion:** a  $1 \times H100$  d24 run is **numerically comparable** to the  $8 \times H100$  run in the statistical sense. Its CORE should land inside the run-to-run noise (about  $\pm 0.008$ , \$2.3). It is not bit-identical, and neither are two 8-GPU runs.

- Log `grad_accum_steps` and the dbs in the paper.
- `val_bpb` uses `eval_steps = eval_tokens // (dbs * seq.world)` ([base\\_train.py L424](#)), so it evaluates the same 41.9M val tokens on any GPU count.

### 3.3 Memory on one 80 GB H100

**Activation checkpointing:** none at HEAD. `gpt.py` has no `torch.utils.checkpoint`; the only "checkpoint" mentions concern checkpoint-file compatibility. Sequence length 2048 ([base\\_train.py L53](#)).

**Static memory for d24, computed from parameter counts and dtypes:**

- **Linear params in fp32** (master weights; the model is built in fp32 and `init_weights` casts only the embeddings to bf16, [gpt.py L262-268](#)):  $729.8M \times 4 B = \mathbf{2.92 GB}$ .
- **Embeddings** (wte + value embeds) in bf16:  $654.3M \times 2 B = \mathbf{1.31 GB}$ .
- **Gradients:** the same dtypes, **4.23 GB**.
- **Optimiser state, unsharded on one GPU:**
  - Muon momentum in fp32 for the matrices: 2.72 GB. The factored second moment is negligible.
  - AdamW  $m$  and  $v$  for `lm_head` (fp32): 0.40 GB.
  - AdamW  $m$  and  $v$  for embeddings (bf16, `zeros_like` of a bf16 param): 2.62 GB.
  - Total:  $\approx \mathbf{5.7 GB}$ . On 8 ranks this is sharded to  $\approx 0.7 GB$  per rank.
- **Total static  $\approx 14.2 GB$**  on one GPU, against  $\approx 9.2 GB$  per rank on 8 GPUs.
- **Transient at optimiser time:** on one rank, Muon stacks all grads plus the owned params,  $\approx 2 \times 2.72 GB \approx \mathbf{5.4 GB}$ . This happens after backward, when activations are freed, so it does not add to the activation peak.

**Activations.** The only primary measurement is #819, in the same model and code family: **21,122 MiB peak at dbs 2** on a 5090, with `--window-pattern L`, SDPA and fp8. Subtracting ~14 GB static leaves ~3.5 GB per 2048-token sequence. Linear extrapolation gives:

- **dbs 16: ≈ 70-75 GB.** This fits in 80 GB, but with little margin.
- **dbs 8: ≈ 42-45 GB.** Safe.

This estimate is UNCERTAIN: it rests on one data point, measured with a different attention kernel (SDPA, not FA3) and the `L` window pattern. The speedrun itself runs dbs 16 on 80 GB cards with ~5 GB less static memory per rank, which is consistent with the estimate. The fp8 activation saving is real: "Memory usage also decreases quite a bit, by ~9GB" at d26 ([LOG](#)).

**Practical rule.** Try `--device-batch-size=16;` on OOM use 8. The global batch, and therefore the maths, is unchanged.

### 3.4 Wall-clock, cost, and interruption risk

**Anchor: 8×H100, 1.65 h pretraining = 13.2 GPU-h.**

- 1.067 s per step × 5,568 steps.
- That is ≈ 8.5 GPU-seconds per step, ≈ 5.9e14 FLOP/s per GPU, ≈ 59% of bf16 peak.
- The 1-GPU run does the same FLOPs with the same micro-batch shape (dbs 16).

**Factors that change the 1-GPU time:**

- **Communication.** No reduce\_scatter/all\_gather on one GPU. This saves a few per cent at most (UNCERTAIN; NCCL overlap was not profiled).
- **Unsharded Muon.** One GPU computes Polar Express for all 156 Muon matrices (144 Linears + 12 `ve_gate`). Estimated ≈ 2.6e13 FLOP per step, ≈ 45 ms against ≈ 8.5 s of forward/backward. Negligible (≈ 0.5%).
- **Evals.** They are excluded from the 1.65 h (`total_training_time` excludes eval). Each val-bpb pass is 41.9M tokens forward-only, ×23, ≈ +5%. The in-training CORE (every 2000 steps, 500 examples per task) also scales ×8 on one GPU.

**Estimate.**

- Pretraining ≈ **13-15 h**.
- End to end, including tokenizer, data download, `base_eval` and `SFT/chat_eval`: ≈ **15-18 h**, scaled from the README's "≈2 hours" on 8×H100 = 16 GPU-h.
- This is UNCERTAIN: no 1×H100 d24 timing was found upstream.
- Cross-check: #819 on one RTX 5090 took 27.1 s per step, 41.4 h, on SDPA with no FA3. That is 3.2× the ~8.5 s per step implied for one H100, a plausible ratio given the 5090's lower dense tensor throughput, but not a calibration.

**Cost.** GPU-hours are roughly conserved, so cost is too. At the README's implied \$3 per GPU-hour (8×H100 at ~\$24/h), 15-18 GPU-h ≈ **\$45-55**. At typical single-H100 on-demand rates of ~\$2-3.5/h ≈ **\$30-65**. These rental rates are UNCERTAIN: market prices, not re-checked. Spot is cheaper but carries the risk below.

**Interruption risk and resume.** A single 15-18 h run is much more exposed to spot pre-emption than a 2 h node run.

- Resume exists: `--save-every N` plus `--resume-from-step S` ([base\\_train.py L70](#), [base\\_train.py L77](#), [base\\_train.py L157-162](#), [base\\_train.py L476-499](#)).

- It saves model, optimiser (per rank, [checkpoint\\_manager.py L41–67](#)), dataloader position and loop state.
- **But:**
  - `speedrun.sh` does not pass `--save-every`, so by default it saves only at the end;
  - the dataloader resume is "approximate" ([dataloader.py L25–41](#)), so a resumed run is not bit-equivalent to an uninterrupted one;
  - optimiser shards are per rank, so resume must use the same world size;
  - each checkpoint is  $\approx 5.5$  GB of model +  $\approx 5.7$  GB of optimiser state.
- **Recommendation:** on spot, add `--save-every 500` and report any resume in the paper.

### 3.5 Evaluation on one GPU

- **CORE.** `evaluate_task` strides examples by rank and `all_reduces` the per-example correctness vector only when `world_size > 1` ([core\\_eval.py L244–261](#)). On one GPU it simply evaluates all examples. The result is **identical in definition** and independent of GPU count: per-example scoring, then the mean.
- **The standalone eval runs on one GPU.** `base_eval.py` documents a plain-python invocation ([base\\_eval.py L14–17](#)).
- **chat\_eval** ([chat\\_eval.py L30–72](#)) and **chat\_sft** (the same grad-accum formula, [chat\\_sft.py L121–127](#)) are also rank-agnostic.
- **Caveat:** in-training CORE uses `--core-metric-max-per-task=500` ([base\\_train.py L75](#)), a subsample. The paper number must come from the standalone `base_eval` (full set). #819 shows the gap: in-training 0.2803 vs standalone 0.2702.

### 3.6 Upstream reports of 1-GPU (or few-GPU) runs

- **README L82** (official): one GPU gives "~identical results ... wait 8 times longer".
- **README L83:** on cards under 80 GB, reduce `--device-batch-size`.
- **#819** (2026-08-02), "GPT-2 grade on a single RTX 5090 in 41 hours":
  - d24, 1.384B params, fp8 (145/158 Linears converted), dbs  $2 \times 256$  accumulation steps, 5,568 steps;
  - 41.4 h at  $\sim 27.1$  s per step; 131 of 170 ClimbMix shards consumed;
  - peak 21,122 MiB;
  - val bpb 0.720161, CORE 0.2702 (standalone) / 0.2803 (in-training).
  - **Blackwell gotcha:** the FA3 loader "succeeds" but fails at run time with "no kernel image is available" on sm\_120. The fix forces the SDPA path, which forces `--window-pattern L`.
  - No maintainer replies.
  - Not comparable to the record: it used the `L` window pattern, not SSSL.
- **Others, listing only (not read):**
  - **#288:** single RTX PRO 6000 Blackwell, Nov 2025;
  - **#212:** single 4090, Oct 2025;
  - **#231:** low-resource single GPU/CPU, Nov 2025.
- **Not found:** no upstream  $1 \times H100$  or  $2/4 \times H100$  d24 timings (UNCERTAIN; based on the discussions search, not exhaustive).

### 3.7 Which GEMMs run in fp8, and in what precision state is kept (pending item, now confirmed from code)

- **The fp8 set.** `--fp8` converts every `nn.Linear` (including nanochat's `Linear` subclass) whose in/out dims are divisible by 16 and  $\geq 128$  ([base\\_train.py L168–192](#)). At d24 that is **145 of 158 Linears**:

- attention `c_q, c_k, c_v, c_proj` ([gpt.py L77-80](#));
- MLP `c_fc, c_proj` ([gpt.py L134-135](#));
- `lm_head` ([gpt.py L177](#)).
- Skipped as too small: `ve_gate` (12→12) and `smear_gate` (24→1).
- **Embeddings (wte, value embeds) are `nn.Embedding` lookups, not GEMMs, so never fp8.**
- **Which GEMMs.** All three per layer are fp8, via nanochat's own `fp8.py` (torchao is no longer used):
  - forward (input and weight in e4m3);
  - grad-input and grad-weight (grad\_output in e5m2).
  - Tensorwise dynamic scaling, through `torch._scaled_mm` with bf16 output ([fp8.py L125-192](#)).
  - The `rowwise` choice in the CLI is UNCERTAIN, since `fp8.py` says "tensorwise dynamic scaling only".
- **State precision. The earlier description holds, with two refinements.**
  - Linear master weights are **fp32** and are cast per matmul ([gpt.py L45-50](#)). Muon momentum is fp32, and the AdamW state for `lm_head` is fp32.
  - **Refinement 1:** embeddings and their AdamW moments are **bf16**, not fp32 ([gpt.py L262-268](#)).
  - **Refinement 2:** activations saved for backward are the **fp8** copies (input\_fp8, weight\_fp8), which is where the ~9 GB saving comes from.
  - The logits are upcast to fp32 for softcap and loss ([gpt.py L511-515](#)).
  - Evaluation (val bpb, CORE, sampling) swaps fp8 modules back to bf16 `Linear` ([base\\_train.py L194-240](#)).
  - The compute dtype is bf16, auto-detected on SM ≥ 80 ([common.py L32](#)).

### 3.8 Corrections made elsewhere in this report by this section

- §B.1: the speedrun trains **5.84B tokens** ( $5,568 \times 2^{20}$ ), not "~7B".
- §1.4: the short sliding window is **512**, not 768 (§3.9).
- §B.3 and §C: fp8 at HEAD is nanochat's own `fp8.py`; **torchao is not a dependency** (it was already dropped on 2026-02-10 per §1.3). The torchao pin was wrong.

### 3.9 Confirmed d24 architecture and parameter budget (read from code, 2026-10-07)

Every number below was read from, or computed by instantiating, the code at 92d63d4 on the meta device. Nothing here is derived from convention.

#### Dimensions.

- `n_embd` = depth × `aspect_ratio` (64), rounded up to a multiple of `head_dim` (128):  $24 \times 64 = \mathbf{1536}$ . `n_head` =  $1536/128 = \mathbf{12}$ , and `n_kv_head` = `n_head` = 12 ([base\\_train.py L129-143](#)).
- Vocab is **32,768** (`tok_train` default, [tok\\_train.py L19](#)), padded to a multiple of 64, which is a no-op at this size ([gpt.py L170](#)).
- Sequence length 2048 ([base\\_train.py L53](#)).
- wte and `lm_head` are **untied**: separate modules, `wte` is `lm_head` is False ([gpt.py L173-177](#)).

**MLP** ([gpt.py L131-141](#)).

- `c_fc` = `Linear(1536 → 6144)`, i.e.  $4 \times n\_embd$ , then `F.relu(x).square()`, then `c_proj` = `Linear(6144 → 1536)`.
- **Two matrices, no gate (no SwiGLU third matrix)**, `bias=False`.
- 18,874,368 params per block ( $8d^2$ ).

**Attention** ([gpt.py L67-128](#)).

- `c_q`, `c_k`, `c_v`, `c_proj`, each  $1536 \times 1536$  with no bias: 9,437,184 per block ( $4d^2$ ).
- QK-norm, RoPE (base 100000).
- `ve_gate` ( $12 \rightarrow 12$  heads, 144 params) on the 12 value-embedding layers.
- Windows: pattern SSSL, final layer forced to full ([gpt.py L287-314](#)).
  - **The short window is 512.** `-(-2048 // 4 // 128) * 128` evaluates to 512, and the instantiated model's `window_sizes` confirm it. The inline comment "(2048 -> 768)" is stale.
  - Full-context (2048) layers: 3, 7, 11, 15, 19, 23 (6 of 24). The other 18 use 512.

**Norms and extras.**

- RMSNorm is `F.rms_norm` with no parameters ([gpt.py L42-43](#)).
- Value embeddings: one `nn.Embedding(32768, 1536)` on every layer with `layer_idx % 2 == (n_layer-1) % 2`, i.e. the 12 odd layers 1...23 ([gpt.py L53-55](#), [gpt.py L189-192](#)).
- Scalars: `resid_lambdas` and `x0_lambdas` (24 each), `smear_gate` ( $24 \rightarrow 1$ ), `smear_lambda`, `backout_lambda` ([gpt.py L178-188](#)).

| Param group ( $d24$ )                               | Count                | Share |
|-----------------------------------------------------|----------------------|-------|
| FFN ( $24 \times 8d^2$ )                            | 452,984,832          | 32.7% |
| Attention q/k/v/o ( $24 \times 4d^2$ )              | 226,492,416          | 16.4% |
| <code>ve_gate</code> ( $12 \times 144$ )            | 1,728                | ~0    |
| <code>wte</code> ( $32,768 \times 1536$ )           | 50,331,648           | 3.6%  |
| <code>lm_head</code> (untied)                       | 50,331,648           | 3.6%  |
| Value embeddings ( $12 \times 32,768 \times 1536$ ) | 603,979,776          | 43.6% |
| Scalars ( $48 + 24 + 1 + 1$ )                       | 74                   | ~0    |
| <b>Total</b>                                        | <b>1,384,122,122</b> |       |

The total equals `sum(p.numel())`, which `num_scaling_params` asserts ([gpt.py L390-417](#)). It differs from #481's 1,384,124,976 (Jan 29) by 2,854. That gap is consistent with later small-module changes; it is not explained line by line (UNCERTAIN).

**The ratio-8 definition** ([base\\_train.py L263-269](#), [base\\_train.py L276-284](#), [base\\_train.py L348-351](#)).

- `get_scaling_params` = `transformer_matrices` + `lm_head` = 679,478,976 + 50,331,648 = **729,810,624**. That is block matrices (including the `ve_gates`) plus `lm_head`. **wte and the value embeddings are excluded.**
- `target_tokens` =  $8 \times 729,810,624 = 5,838,484,992$ .
- The auto batch is  $2^{\text{round}(\log_2(2^{19} \cdot (D/D_{\text{ref}})^{0.383}))} = 2^{20}$ , with  $D_{\text{ref}} = 8 \times$  the  $d12$  scaling params = 880,807,296.
- `num_iterations` = `target_tokens // 2^20` = 5,568, so **5,838,471,168 tokens**.
- Per *total* param that is 4.22 tokens; per block matrix (FFN + attention) 8.59.

**Reproducing  $d20 = 560,988,160$**  (launch, #1).

- $12 \cdot L \cdot d^2 + 2 \cdot V \cdot d$  with  $L = 20$ ,  $d = 1280$  and  $V = 65,536$  (the launch tokenizer) gives exactly 560,988,160. The launch model had no value embeddings, gates or scalars.

- The same d20 under HEAD code would be 896,533,746 total ( $V = 32,768$ , plus value embeddings).

**FLOP split per token** (`estimate_flops` =  $6 \times \text{matmul params} + \sum 12 \cdot h \cdot q \cdot \text{window}$ , [gpt.py L319-349](#)).

- Total **4.775e9**.
- Per layer, FFN against attention projections is exactly  $\frac{2}{3}$  ( $8d^2$  vs  $4d^2$ ).
- Including attention-score FLOPs, the FFN share is **63.2%** in the 512-window layers and **54.5%** in the full layers, and **60.8%** over all blocks.
- Of the whole per-token training FLOPs:
  - FFN 56.9%;
  - attention projections 28.5%;
  - attention scores 8.3%;
  - `lm_head` 6.3%. Embedding lookups count zero.